# Methods Overview

# Methods Overview I

The pipeline is a sequence of deterministic stages, each reading the previous stage's committed artifacts and writing its own. Business logic lives in tested `src/` modules; the numbered `scripts/` are thin orchestrators that wire I/O, configuration loading, logging, and stage sequencing. The architecture follows the thin orchestrator pattern: no computational logic resides in scripts.

# Pipeline Stages I

1. **Retrieval** (`01_literature_search.py`) — dispatch the configured query across 10 engines (arXiv, OpenAlex, Semantic Scholar, Crossref, PubMed, SovietRxiv, ChinaRxiv, Europe PMC, bioRxiv, and medRxiv), merge, and de-duplicate into `corpus.jsonl`. Each engine is an isolated adapter exposing a uniform `search(query) -> list[Paper]` interface; engines that are keyless need no credentials, while Semantic Scholar uses a key when present. SovietRxiv and ChinaRxiv share a unified API with an optional `X-API-Email` header for the polite rate-limit pool (300/min vs 30/min anonymous).

# Pipeline Stages II

2. **Meta-analysis** (`02_meta_analysis_pipeline.py`) — subfield classification, temporal metrics, TF-IDF, non-negative matrix factorization topics, and the citation network. This stage reads `corpus.jsonl` and emits `subfield_classification.json`, `temporal_analysis.json`, `tfidf_data.json`, `topics.json`, `citation_network.json`, and `citation_graph.gml`.
3. **Knowledge graph** (`03_build_knowledge_graph.py`, optional/LLM-gated) — extract assertions and score the 6 configured hypotheses. Outputs `nanopublications.jsonl`, `hypothesis_scores.json`, and `assertion_summary.json`.
4. **Figures** (`04_generate_figures.py`) — render 21 publication-ready visualizations from the analysis JSON outputs. All figures use a colourblind-safe palette (Wong 2011), high-contrast labels at $\geq 16$pt, and a headless matplotlib backend (Agg).

# Pipeline Stages III

5. **Injection** (`05_inject_variables.py`) — compute manuscript variables from the artifacts above and substitute them into these Markdown sections. An unresolved placeholder is a hard error, not a silent gap.
6. **Fulltext assessment** (`06_fulltext_assessment.py`) — report abstract coverage (61.6%), open-access status (24.6%), and PDF availability (54.6%) across the corpus.

# Reproducibility Model I

The system runs **offline and deterministically** by default: a committed synthetic seed corpus drives every stage with fixed seeds (seed $= 42$ for NMF, SVD, and graph layouts), so re-running produces byte-identical outputs. A live run with engines enabled and credentials supplied replaces the seed corpus with real records — as in this instance, which retrieved 2334 live records. The template is domain-agnostic: the search term, query, keyword set, subfield taxonomy, and hypotheses all come from `manuscript/config.yaml`.

# Configuration Surface I

A single `manuscript/config.yaml` controls:

- **Search parameters**: term, query string, per-engine queries, relevance keywords, start year, max results, resume/clear behaviour
- **Engine toggles**: arXiv, OpenAlex, Semantic Scholar, Crossref, PubMed, SovietRxiv, ChinaRxiv, Europe PMC, bioRxiv, and medRxiv (each independently enabled or disabled)
- **SovietRxiv/ChinaRxiv settings**: optional `api_email` for the polite pool, `source` filter (`russiarxiv` or `chinaxiv`)
- **Full-text download**: opt-in Unpaywall resolution with `unpaywall_email`
- **Embeddings**: method (`tfidf_svd` or `transformer`), dimensionality, max features
- **Knowledge graph**: checkpoint interval, LLM model, base URL, temperature, max tokens

# Configuration Surface II

- **Hypothesis definitions**: 6 named hypotheses with scope labels
- **Subfield taxonomy**: 6 buckets, each with a keyword list
- **Paper metadata**: title, authors, DOI, keywords, license, repository URL